

Лабораторна 7

Нелінійні найменші квадрати, групи Лі та GTSAM

Покроковий гайд з виконання: теорія, математика, код, результати

Що всередині:

- MAP-оцінка $\rightarrow$ нелінійні найменші квадрати; Гаусс–Ньютон і Левенберг–Марквардт;
- оптимізація на многовидах $SO(3)$ / $SE(3)$: експонента, ретракція, якобіани в GTSAM;
- індивідуальні Deliverables 1–2: зважені НМК, хордальне усереднення поз, доведення ML для Langevin + Gauss, кроки GN/LM з явними якобіанами;
- командні Deliverables 1–5: перевірений код для GTSAM 4.2 (Docker-образ курсу) і реальні результати прогонів;
- пастки: порядок координат у `Pose3`, калібрувальна свобода (gauge), переплутані конфіги RViz.

Зміст

1	Огляд лабораторної	3
1.1	Позначення	3
2	Математичне підґрунтя	4
2.1	Від MAP до нелінійних найменших квадратів	4
2.2	Гаусс–Ньютон і Левенберг–Марквардт	4
2.3	Оптимізація на многовидах: $SO(3)$ і $SE(3)$	5
2.4	Калібрувальна свобода (gauge)	5
2.5	Шумові моделі та «вибілювання» в GTSAM	5
3	Індивідуальний Deliverable 1 — коваріації в НМК	6
3.1	(1) Діагональна W	6
3.2	(2) Довільна додатно визначена W	6
3.3	(3) Нормальні рівняння	6
4	Індивідуальний Deliverable 2 — групи Лі	7
4.1	(1) Хордальне середнє в $SE(3)$	7
4.2	(2) Зважене хордальне середнє	7
4.3	(3) Це ML-оцінка (Langevin + Gauss)	8
4.4	(4) НМК і кроки Гаусса–Ньютона / Левенберга–Марквардта ($\omega_i = 1$)	8
4.5	(5) Явні якобіани (+5)	9
5	Підготовка середовища	10
6	Командний Deliverable 1 — вступ до GTSAM	11
6.1	Теорія	11
6.2	Код	11
6.3	Результат (реальний прогін)	12
7	Командний Deliverable 2 — 3D pose graph	13
7.1	Що генерує стенсил	13
7.2	Код	13
7.3	Результати (реальні прогони, без MoCap)	13
8	Командний Deliverable 3 — фактор Motion Capture	15
8.1	Виведення фактора	15
8.2	Код 3a — <code>include/deliverable_2_3.h</code>	15
8.3	Код 3b — <code>src/deliverable_2_3.cpp</code>	16
8.4	Результати (реальні прогони, з MoCap)	16
9	Командний Deliverable 4 — bundle adjustment	17
9.1	Теорія	17
9.2	Код	17
9.3	Результати (реальний прогін)	18
10	Командний Deliverable 5 (опційний) — усереднення $SO(3)$	19
10.1	Теорія	19
10.2	Код — <code>include/deliverable_5.h</code>	19
10.3	Код — <code>src/deliverable_5.cpp</code>	19
10.4	Результати (реальний прогін)	20
11	Чекліст здачі	21
A	Словник позначень	22

1 Огляд лабораторної

Lab 7 — про те, як **формулювати** задачі оцінювання стану у вигляді нелінійних найменших квадратів (НМК) і розв’язувати їх бібліотекою GTSAM. Усі наступні лаби (VIO, SLAM) будуються саме на цьому.

№	Завдання	Тип	Бали
I-1	Коваріації вимірювань у НМК: зважені НМК $\rightarrow$ стандартні, нормальні рівняння	індивідуальне	10
I-2	Групи Лі: хордальне усереднення $SE(3)$, зважена версія, доведення ML, кроки GN/LM (+5 за якобіани)	індивідуальне	35
T-1	Вступ до GTSAM: Pose2, odometry + «GPS»	командне	10
T-2	3D pose graph: BetweenFactor<Pose3> + prior, 1/3/6 ітерацій GN	командне	10
T-3	Власний фактор MoCap (позиція $\in \mathbb{R}^3$) з якобіаном	командне	15
T-4	Bundle adjustment: GenericProjectionFactor	командне	20
T-5	(опц.) Усереднення $SO(3)$: власний Frobenius-фактор	командне	+15

Файли (VNAV-labs/lab7/):

src/deliverable_1.cpp	T-1: UnaryFactor (вже написаний) + TODO 1a/1b/1c
include/deliverable_2_3.h	T-3: MoCapPosition3Factor (TODO 3a)
src/deliverable_2_3.cpp	T-2/T-3: генерація даних, TODO 2a/2b/3b, візуалізація
src/deliverable_4.cpp, include/example_data.h	T-4: камери навколо куба, TODO — фактори й priors
include/deliverable_5.h, src/deliverable_5.cpp	T-5: Frobenius-фактор і оптимізація
launch/*.launch.yaml, rviz/*.rviz	запуск вузлів + RViz2

1.1 Позначення

- $x \in \mathbb{R}^n$ — вектор невідомих (для евклідових задач); X — набір усіх змінних факторного графа (пози, точки).
- $r(x) \in \mathbb{R}^m$ — вектор залишків (residuals); $J = \partial r / \partial x \in \mathbb{R}^{m \times n}$ — якобіан.
- $\|e\|_{\Sigma}^2 = e^T \Sigma^{-1} e$ — квадрат **махаланобісової** норми з коваріацією Σ .
- $R \in SO(3)$ — обертання, $T = \begin{pmatrix} R & t \\ 0 & 1 \end{pmatrix} \in SE(3)$ — поза. $[\omega]_{\times}$ — косиметрична матриця векторного добутку; e_k — k -й базисний вектор $\mathbb{R}^3$.
- $\text{Exp} : \mathbb{R}^3 \rightarrow SO(3)$, Log — експонента й логарифм групи («великими літерами» — з урахуванням $(\cdot)^\wedge / (\cdot)^\vee$).
- $\text{vec}(M)$ — матриця, записана в стовпець (по стовпцях), $\|M\|_F$ — норма Фробеніуса, tr — слід.
- Після ключових формул — блок «**де:**» з поясненням змінних; у кінці — словник позначень (додаток A).

2 Математичне підґрунтя

2.1 Від MAP до нелінійних найменших квадратів

Маємо невідомі X і вимірювання $z_1, \dots, z_K$. MAP-оцінка:

$$X_{\text{MAP}} = \arg \max_X p(X | Z) = \arg \max_X p(Z | X)p(X) = \arg \max_X \prod_i p(z_i | X_i) \cdot p(X). \quad (1)$$

Якщо кожне вимірювання — нелінійна функція частини змінних плюс гаусів шум, $z_i = h_i(X_i) + \varepsilon_i$, $\varepsilon_i \sim \mathcal{N}(0, \Sigma_i)$, то

$$p(z_i | X_i) = \frac{1}{\sqrt{(2\pi)^{d_i} \det \Sigma_i}} \exp\left(-\frac{1}{2}\|h_i(X_i) - z_i\|_{\Sigma_i}^2\right). \quad (2)$$

Логарифм — монотонна функція, тож максимізація добутку = мінімізація суми від’ємних логарифмів; нормувальні множники від X не залежать:

$$X_{\text{MAP}} = \arg \min_X \sum_i \|h_i(X_i) - z_i\|_{\Sigma_i}^2. \quad (3)$$

де:

X, X_i усі невідомі / підмножина змінних, від якої залежить i -те вимірювання
 z_i i -те вимірювання (відносна поза, GPS-позиція, піксель тощо)
 h_i модель вимірювання: яке z_i ми **очікуємо** побачити при значенні X_i
 ε_i, Σ_i шум вимірювання та його коваріація ($d_i \times d_i$)
 d_i розмірність i -го вимірювання
 $p(X)$ апіорний розподіл (у GTSAM — prior-фактори, тобто теж доданки в сумі)

Кожен доданок 3 — **фактор**. Сукупність змінних і факторів — **факторний граф**: вершини-змінні, вершини-фактори, ребра «фактор залежить від змінної». GTSAM зберігає саме таку структуру (NonlinearFactorGraph) і використовує її розпріженість.

2.2 Гаусс–Ньютон і Левенберг–Марквардт

Розглянемо $\min_x f(x) = \|r(x)\|^2$. Лінеаризуємо залишок у поточній точці: $r(x + \delta) \approx r(x) + J\delta$. Тоді

$$f(x + \delta) \approx \|r + J\delta\|^2 = r^\top r + 2\delta^\top J^\top r + \delta^\top J^\top J\delta. \quad (4)$$

Похідна за δ дорівнює нулю, звідки **нормальні рівняння** Гаусса–Ньютона:

$$J^\top J\delta = -J^\top r, \quad x \leftarrow x + \delta. \quad (5)$$

Левенберг–Марквардт додає демпфування:

$$(J^\top J + \lambda D)\delta = -J^\top r, \quad (6)$$

де $D = I$ або $D = \text{diag}(J^\top J)$. При $\lambda \rightarrow 0$ — крок GN (швидка квадратична збіжність біля мінімуму); при великому λ — короткий крок уздовж антиградієнта $-J^\top r/\lambda$ (надійно далеко від мінімуму). Якщо крок зменшив f — приймаємо і зменшуємо λ (наприклад, у 10 разів), інакше відкидаємо і збільшуємо λ .

де:

$f(x)$ цільова функція — сума квадратів залишків
 δ крок (поправка) до поточного значення x
 $J^\top J$ наближення Гаусса–Ньютона до гессіана f (без других похідних r)
 $-J^\top r$ антиградієнт f (з точністю до множника 2)
 λ параметр демпфування LM (у GTSAM — lambdaInitial, lambdaFactor)
 D матриця масштабування демпфування

2.3 Оптимізація на многовидах: $SO(3)$ і $SE(3)$

Пози й обертання не є векторами: $R + \delta$ — не обертання. Тому крок робимо в **дотичному просторі** і «накручуємо» його на групу **ретракцією**. GTSAM використовує праве збурення:

$$R \oplus \omega = R \operatorname{Exp}(\omega), \quad T \oplus \xi = T \operatorname{Exp}(\xi), \quad \xi = \begin{pmatrix} \omega \\ v \end{pmatrix} \in \mathbb{R}^6. \quad (7)$$

Для малих приростів $\operatorname{Exp}(\omega) \approx I + [\omega]_{\times}$, а для пози

$$T \operatorname{Exp}(\xi) \approx (R(I + [\omega]_{\times}), t + Rv). \quad (8)$$

Якобіан на многовиді — похідна за локальними координатами в нулі:

$$H = \left. \frac{\partial e(X \oplus \delta)}{\partial \delta} \right|_{\delta=0}. \quad (9)$$

Саме цей H повертає `evaluateError` у GTSAM. Після розв'язку 5 чи 6 оновлення — $X \leftarrow X \oplus \delta$.

де:

$\omega \in \mathbb{R}^3$	мале обертання (вісь $\times$ кут, радіани) у локальній системі
$v \in \mathbb{R}^3$	малий зсув у локальній системі пози (тому в $\mathbb{R}^3$ світу він стає Rv)
ξ	локальні координати $SE(3)$; у GTSAM спершу обертання, потім трансляція
Exp	експонента групи: $\operatorname{Exp}(\omega) = \exp([\omega]_{\times})$ (формула Родріга)
H	якобіан похибки фактора за локальними координатами змінної

Пастка

Порядок $[\omega; v]$ у `Pose3` — найчастіша помилка. Сигми шуму для `Pose3` задаються як `(rot, rot, rot, pos, pos, pos)` — стенси́л так і робить. Якщо переплутати, похибка 1 см трактуватиметься як 1 рад.

2.4 Калібрувальна свобода (gauge)

Якщо всі фактори **відносні** (одометрія між позами, проєкції), то зсув/поворот усього розв'язку як цілого не змінює жодної похибки. Тоді $J^T J$ вироджена (має ядро), і система 5 не має єдиного розв'язку. Цю свободу знімають **абсолютні** фактори: ріг на першу позу, GPS/МоСар-вимірювання. У `bundle adjustment` свобода 7-вимірна: 6 (жорстке перетворення) + 1 (масштаб).

2.5 Шумові моделі та «вибілювання» в GTSAM

GTSAM мінімізує

$$E(X) = \frac{1}{2} \sum_i \|e_i(X)\|_{\Sigma_i}^2 = \frac{1}{2} \sum_i \left\| \Sigma_i^{-1/2} e_i(X) \right\|^2, \quad (10)$$

тобто кожен залишок множиться на $\Sigma_i^{-1/2}$ («whitening»), після чого задача стає звичайними НМК. `graph.error(values)` повертає саме $E(X)$ — з множителем 1/2.

- `noiseModel::Diagonal::Sigmas( $\sigma$ )` — $\Sigma = \operatorname{diag}(\sigma_1^2, \dots)$;
- `noiseModel::Isotropic::Sigma( $d, \sigma$ )` — $\Sigma = \sigma^2 I_d$;
- `noiseModel::Gaussian::Covariance( $\Sigma$ )` — повна коваріація.

Це і є індивідуальний Deliverable 1 «в коді».

3 Індивідуальний Deliverable 1 — коваріації в НМК

Individual Deliverable 1 (10 балів)

Цільова функція $f(x) = r(x)^\top W r(x)$, $r : \mathbb{R}^n \rightarrow \mathbb{R}^m$, W — додатно визначена $m \times m$. (1) Звести до стандартних НМК при діагональній W ; (2) при недиагональній W ; (3) вивести нормальні рівняння Гаусса–Ньютона для $\min r^\top W r$.

3.1 (1) Діагональна W

$W = \text{diag}(w_1, \dots, w_m)$, $w_j > 0$. Тоді

$$f(x) = \sum_{j=1}^m w_j r_j(x)^2 = \sum_{j=1}^m (\sqrt{w_j} r_j(x))^2 = \|\tilde{r}(x)\|^2, \quad \tilde{r}(x) = W^{1/2} r(x), \quad (11)$$

де $W^{1/2} = \text{diag}(\sqrt{w_1}, \dots, \sqrt{w_m})$. Тобто кожен залишок просто масштабується на $\sqrt{w_j}$.

де:

$r_j(x)$ j -та компонента вектора залишків
 w_j вага j -го залишку; в імовірнісній постановці $w_j = 1/\sigma_j^2$
 $\tilde{r}$ «віблений» залишок — для нього задача стандартна

3.2 (2) Довільна додатно визначена W

Будь-яку симетричну додатно визначену W можна розкласти (Холецький): $W = LL^\top$, L — нижньотрикутна з додатною діагоналлю. Тоді

$$f(x) = r^\top LL^\top r = (L^\top r)^\top (L^\top r) = \|L^\top r(x)\|^2, \quad \tilde{r}(x) = L^\top r(x). \quad (12)$$

Альтернатива — симетричний квадратний корінь $W^{1/2} = Q\Lambda^{1/2}Q^\top$ з власного розкладу $W = Q\Lambda Q^\top$: $f = \|W^{1/2}r\|^2$. Обидва варіанти дають ту саму задачу; Холецький дешевший. Якщо $W = \Sigma^{-1}$, то $\tilde{r} = L^\top r$ — це саме «whitening» GTSAM: компоненти $\tilde{r}$ некорельовані й мають одиничну дисперсію.

де:

L множник Холецького: $W = LL^\top$
 Q, Λ власні вектори (ортогональна матриця) і власні числа W
 Σ коваріація вимірювань; найкраща вага — $W = \Sigma^{-1}$

3.3 (3) Нормальні рівняння

Лінеаризація: $r(x + \delta) \approx r + J\delta$, $J = \partial r / \partial x|_x$. Підставляємо:

$$f(x + \delta) \approx (r + J\delta)^\top W (r + J\delta) = r^\top W r + 2\delta^\top J^\top W r + \delta^\top J^\top W J \delta \quad (13)$$

(використано $W = W^\top$). Градієнт за δ : $2J^\top W r + 2J^\top W J \delta = 0$, звідки

$$J^\top W J \delta = -J^\top W r(x). \quad (14)$$

$J^\top W J$ додатно визначена, якщо J має повний стовпцевий ранг, тож крок єдиний. Перевірка: з $\tilde{r} = L^\top r$, $\tilde{J} = L^\top J$ стандартні рівняння $\tilde{J}^\top \tilde{J} \delta = -\tilde{J}^\top \tilde{r}$ збігаються з 14, бо $\tilde{J}^\top \tilde{J} = J^\top LL^\top J = J^\top W J$.

де:

J якобіан r у поточній точці, $m \times n$
 $J^\top W J$ зважене наближення гессіана (в GTSAM — «інформаційна матриця» лінеаризованої задачі)
 $\tilde{J} = L^\top J$ віблений якобіан

4 Індивідуальний Deliverable 2 — групи ЛІ

Individual Deliverable 2 (35 балів)

Дано $T_i \in \text{SE}(3)$, $i = 1..n$. (1) Замкнена формула для $T^* = \arg \min_{T \in \text{SE}(3)} \frac{1}{n} \sum_i \|T - T_i\|_F^2$. (2) Зважена версія з $\|M\|_\Omega^2 = \text{tr}(M\Omega M^\top)$, $\Omega_i = \text{diag}(\omega_i I_3, \rho_i)$; вплив ваг. (3) Довести, що це ML-оцінка для $R_i \sim \text{Langevin}(R, \omega_i)$, $t_i \sim \mathcal{N}(t, \rho_i^{-1} I)$. (4) Записати як НМК при $\omega_i = 1$, вивести кроки GN і LM. (5, +5) Явні якобіани.

4.1 (1) Хордальне середнє в $\text{SE}(3)$

Розщеплення. $T - T_i = \begin{pmatrix} R - R_i & t - t_i \\ \mathbf{0}^\top & 0 \end{pmatrix}$ — нижній рядок зникає. Норма Фробеніуса — сума квадратів усіх елементів, тож

$$\|T - T_i\|_F^2 = \|R - R_i\|_F^2 + \|t - t_i\|^2. \quad (15)$$

Задача розпадається на незалежні задачі для $t \in \mathbb{R}^3$ і $R \in \text{SO}(3)$ (множник $1/n$ на $\arg \min$ не впливає).

Трансляція. $\nabla_t \sum_i \|t - t_i\|^2 = 2 \sum_i (t - t_i) = 0$:

$$t^* = \frac{1}{n} \sum_{i=1}^n t_i. \quad (16)$$

Обертання. Розкриваємо: $\|R - R_i\|_F^2 = \text{tr}(R^\top R) - 2 \text{tr}(R^\top R_i) + \text{tr}(R_i^\top R_i) = 6 - 2 \text{tr}(R^\top R_i)$. Отже

$$R^* = \arg \max_{R \in \text{SO}(3)} \text{tr}(R^\top M), \quad M = \sum_{i=1}^n R_i. \quad (17)$$

Нехай $M = USV^\top$ (SVD, $S = \text{diag}(s_1, s_2, s_3)$, $s_1 \geq s_2 \geq s_3 \geq 0$). Тоді $\text{tr}(R^\top USV^\top) = \text{tr}(V^\top R^\top US) = \text{tr}(ZS) = \sum_k Z_{kk} s_k$, де $Z = V^\top R^\top U$ ортогональна, отже $Z_{kk} \leq 1$. Максимум при $Z = I$, тобто $R = UV^\top$. Якщо $\det(UV^\top) = -1$, це відбиття; найкраще обертання тоді отримуємо, «жертвуючи» найменшим сингулярним числом:

$$R^* = U \text{diag}(1, 1, \det(UV^\top)) V^\top = \mathcal{P}_{\text{SO}(3)} \left(\sum_i R_i \right). \quad (18)$$

Тобто хордальне середнє обертань = проєкція арифметичного середнього матриць на $\text{SO}(3)$.

де:

T^*, R^*, t^*	шукане середнє та його частини
M	сума матриць обертання (сама вона, загалом, не обертання)
U, S, V	SVD матриці M
Z	допоміжна ортогональна матриця в доведенні
$\mathcal{P}_{\text{SO}(3)}$	проєкція на $\text{SO}(3)$ — найближче за Фробеніусом обертання

4.2 (2) Зважене хордальне середнє

$M = T - T_i = \begin{pmatrix} \Delta R & \Delta t \\ \mathbf{0}^\top & 0 \end{pmatrix}$, $\Omega_i = \begin{pmatrix} \omega_i I_3 & \mathbf{0} \\ \mathbf{0}^\top & \rho_i \end{pmatrix}$. Тоді

$$M\Omega_i M^\top = \begin{pmatrix} \omega_i \Delta R \Delta R^\top + \rho_i \Delta t \Delta t^\top & \mathbf{0} \\ \mathbf{0}^\top & 0 \end{pmatrix}, \quad \text{tr}(M\Omega_i M^\top) = \omega_i \|R - R_i\|_F^2 + \rho_i \|t - t_i\|^2. \quad (19)$$

Задача знову розпадається, і ті самі міркування дають

$$\mathbf{t}^* = \frac{\sum_i \rho_i \mathbf{t}_i}{\sum_i \rho_i}, \quad R^* = \mathcal{P}_{\text{SO}(3)} \left(\sum_i \omega_i R_i \right). \quad (20)$$

Вплив ваг. ω_i впливає **лише** на обертання, ρ_i — **лише** на трансляцію. Якщо для одного i обидва коефіцієнти дуже великі ($\omega_i, \rho_i \rightarrow \infty$), то $\sum_j \omega_j R_j \approx \omega_i R_i$ і $\mathbf{t}^* \rightarrow \mathbf{t}_i$: середнє «прилипає» до T_i (це вимірювання вважається майже точним). Якщо велике лише ω_i — до R_i прилипає тільки обертання, а трансляція лишається зваженим середнім інших.

де:

ω_i	вага (концентрація) обертальної частини i -го вимірювання
ρ_i (у завданні ϱ_i)	вага трансляційної частини, обернена дисперсія
$\Delta R, \Delta \mathbf{t}$	$R - R_i$ та $\mathbf{t} - \mathbf{t}_i$

4.3 (3) Це ML-оцінка (Langevin + Gauss)

Розподіл Ланжевена (фон Мізеса–Фішера для матриць) з модою R і концентрацією ω :

$$p(R_i | R) = \frac{1}{c(\omega_i)} \exp(\omega_i \text{tr}(R^\top R_i)). \quad (21)$$

Гаусів розподіл трансляції з коваріацією $\rho_i^{-1}I$:

$$p(\mathbf{t}_i | \mathbf{t}) = \left(\frac{\rho_i}{2\pi} \right)^{3/2} \exp\left(-\frac{\rho_i}{2} \|\mathbf{t}_i - \mathbf{t}\|^2 \right). \quad (22)$$

Вимірювання незалежні, і R_i не залежить від $\mathbf{t}$, тож

$$-\log \prod_i p(T_i | T) = \sum_i \left[-\omega_i \text{tr}(R^\top R_i) + \frac{\rho_i}{2} \|\mathbf{t} - \mathbf{t}_i\|^2 \right] + C, \quad (23)$$

де C не залежить від T (нормувальні константи $c(\omega_i)$ залежать лише від ω_i). Використаємо тотожність з пункту (1): $\text{tr}(R^\top R_i) = 3 - \frac{1}{2} \|R - R_i\|_F^2$. Тоді

$$-\log \prod_i p(T_i | T) = \frac{1}{2} \sum_i [\omega_i \|R - R_i\|_F^2 + \rho_i \|\mathbf{t} - \mathbf{t}_i\|^2] + C' = \frac{1}{2} \sum_i \|T - T_i\|_{\Omega_i}^2 + C'. \quad (24)$$

Максимізація правдоподібності $\Leftrightarrow$ мінімізація $-\log \Leftrightarrow$ мінімізація $\frac{1}{n} \sum_i \|T - T_i\|_{\Omega_i}^2$ (додатні множники $1/2$, $1/n$ і константа C' не змінюють $\arg \min$). $\square$

де:

$c(\omega)$	нормувальна константа Ланжевена (інтеграл по $\text{SO}(3)$); не залежить від R
C, C'	константи, що не залежать від T

Примітка

Якщо у вашому конспекті Ланжевен записано як $\exp(\omega/2 \cdot \text{tr}(R^\top R_i))$, доведення те саме, лише в Ω_i з'явиться $\omega_i/2$ замість ω_i — на форму розв'язку це не впливає.

4.4 (4) НМК і кроки Гаусса–Ньютона / Левенберга–Марквардта ($\omega_i = 1$)

Залишки. Для кожного вимірювання — 12-вимірний вектор

$$\mathbf{r}_i(T) = \begin{pmatrix} \text{vec}(R - R_i) \\ \sqrt{\rho_i}(\mathbf{t} - \mathbf{t}_i) \end{pmatrix} \in \mathbb{R}^{12}, \quad \min_T \sum_i \|\mathbf{r}_i(T)\|^2. \quad (25)$$

Параметризація кроку — ретракція 7, $\xi = (\omega, \mathbf{v})$. З 8:

$$r_i(T \oplus \xi) \approx \left(\frac{\text{vec}(R - R_i + R[\omega]_{\times})}{\sqrt{\rho_i}(t - t_i + Rv)} \right) = r_i(T) + J_i \xi. \quad (26)$$

Оскільки $[\omega]_{\times} = \sum_k \omega_k [e_k]_{\times}$ (лінійно по ω),

$$J_i = \begin{pmatrix} J_R & 0_{9 \times 3} \\ 0_{3 \times 3} & \sqrt{\rho_i} R \end{pmatrix}, \quad J_R = \begin{pmatrix} \text{vec}(R[e_1]_{\times}) & \text{vec}(R[e_2]_{\times}) & \text{vec}(R[e_3]_{\times}) \end{pmatrix} \in \mathbb{R}^{9 \times 3}. \quad (27)$$

Крок Гаусса–Ньютона. $(\sum_i J_i^{\top} J_i) \xi = -\sum_i J_i^{\top} r_i$. Блоки обчислюються явно:

- $J_R^{\top} J_R = 2I_3$: стовпці ортогональні, $\|R[e_k]_{\times}\|_F^2 = \| [e_k]_{\times} \|_F^2 = 2$;
- $J_R^{\top} \text{vec}(R) = 0$: $\langle R[e_k]_{\times}, R \rangle = \text{tr}([e_k]_{\times}^{\top}) = 0$;
- $J_R^{\top} \text{vec}(R_i) = w(A_i)$, де $A_i = R^{\top} R_i$ і $w(A) = (A_{32} - A_{23}, A_{13} - A_{31}, A_{21} - A_{12})^{\top} = 2 \text{vee}(\text{skew}(A))$, $\text{skew}(A) = (A - A^{\top})/2$. (Покомпонентно: $\langle R[e_k]_{\times}, R_i \rangle = \text{tr}([e_k]_{\times}^{\top} R^{\top} R_i)$, а $[e_1]_{\times}$ має +1 у позиції (3, 2) і -1 у (2, 3) — звідси $A_{32} - A_{23}$.)

Звідси кроки (обертання й трансляція незалежні — матриця блочно-діагональна):

$$\omega_{\text{GN}} = \frac{1}{2n} \sum_i w(R^{\top} R_i) = \frac{1}{n} \sum_i \text{vee}(\text{skew}(R^{\top} R_i)), \quad v_{\text{GN}} = R^{\top}(\bar{t}_{\rho} - t), \quad \bar{t}_{\rho} = \frac{\sum_i \rho_i t_i}{\sum_i \rho_i}. \quad (28)$$

Оновлення: $R \leftarrow R \text{Exp}(\omega_{\text{GN}})$, $t \leftarrow t + Rv_{\text{GN}} = \bar{t}_{\rho}$.

Інтуїція. Для малого кута $\text{vee}(\text{skew}(\text{Exp}(\theta))) = \sin\|\theta\| \cdot \theta/\|\theta\| \approx \theta$, тож $\omega_{\text{GN}} \approx \frac{1}{n} \sum_i \text{Log}(R^{\top} R_i)$ — середнє «відхилень» вимірювань від поточної оцінки в дотичному просторі. Трансляційна частина лінійна, тому GN знаходить її точно за **один** крок.

Крок Левенберга–Марквардта. $(H + \lambda D)\xi = -g$, де $H = \text{diag}(2nI_3, (\sum_i \rho_i)I_3)$, $g = \sum_i J_i^{\top} r_i$. $D = \text{diag}(H)$ (Марквардт):

$$\omega_{\text{LM}} = \frac{1}{1 + \lambda} \omega_{\text{GN}}, \quad v_{\text{LM}} = \frac{1}{1 + \lambda} v_{\text{GN}}. \quad (29)$$

З $D = I$: $\omega_{\text{LM}} = 2n/(2n + \lambda) \cdot \omega_{\text{GN}}$, $v_{\text{LM}} = (\sum \rho_i)/(\sum \rho_i + \lambda) \cdot v_{\text{GN}}$. Правило: якщо $\sum \|r_i\|^2$ зменшилась — приймаємо крок і $\lambda \leftarrow \lambda/10$, інакше відкидаємо і $\lambda \leftarrow 10\lambda$.

де:

$r_i(T)$	залишок i -го вимірювання: 9 компонент обертання + 3 трансляції
J_i	якобіан r_i за ξ у точці $\xi = 0$, 12×6
J_R	якобіан $\text{vec}(R \text{Exp}(\omega))$ за ω
$A_i = R^{\top} R_i$	відносне обертання вимірювання щодо поточної оцінки
$w(A)$, vee , skew	вектор з антисиметричної частини A ; vee — обернена до $[\cdot]_{\times}$
g , H	градієнт (без множника 2) і наближений гессіан

4.5 (5) Явні якобіани (+5)

Позначимо стовпці $R = [r_1 \mid r_2 \mid r_3]$. Стовпці $[e_k]_{\times}$: для $k = 1$ — $(0, e_3, -e_2)$, для $k = 2$ — $(-e_3, 0, e_1)$, для $k = 3$ — $(e_2, -e_1, 0)$. Множимо на R зліва:

$$R[e_1]_{\times} = [0 \mid r_3 \mid -r_2], \quad R[e_2]_{\times} = [-r_3 \mid 0 \mid r_1], \quad R[e_3]_{\times} = [r_2 \mid -r_1 \mid 0]. \quad (30)$$

Отже (вес — по стовпцях, блоки 3×1):

$$J_R = \begin{pmatrix} 0 & -r_3 & r_2 \\ r_3 & 0 & -r_1 \\ -r_2 & r_1 & 0 \end{pmatrix}, \quad \frac{\partial \sqrt{\rho_i}(t \oplus v - t_i)}{\partial v} = \sqrt{\rho_i} R, \quad \frac{\partial}{\partial \omega}(\dots) = 0. \quad (31)$$

Цей самий J_R реалізовано в командному Deliverable 5 і перевірено числово (результат GTSAM збігся з замкненою формулою до 10^{-12} рад).

5 Підготовка середовища

```
bash ros2-docker/run_lab_dev.sh lab7 --build-only # збирання lab_7 у Docker
bash ros2-docker/run_lab_dev.sh lab7             # + запуск Deliverable 2/3 з RViz2
```

Всередині контейнера:

```
source /opt/ros/humble/setup.bash && cd /workspace
colcon build --symlink-install --packages-select lab_7 && source install/setup.bash
ros2 run lab_7 deliverable_1
ros2 launch lab_7 deliverable_2_3.launch.yaml max_solver_iterations:=3 use_mocap:=false
ros2 launch lab_7 deliverable_4.launch.yaml
ros2 run lab_7 deliverable_5
```

Версія GTSAM в образі — 4.2a9 (ros-humble-gtsam). Це важливо: у ній `evaluateError` ще приймає `boost::optional<Matrix&> H`, як у стенсилі. У GTSAM 4.3+ сигнатура інша (`OptionalMatrixType H`), і код з інтернету під новішу версію тут не збереться.

Пастка

У поточній версії `include/deliverable_2_3.h` у репозиторії конструктор передає в базовий клас `poseclKey` — такої змінної немає, тож `lab_7` не збереться, доки там не буде `poseKey` (як у коді нижче).

Пастка

RViz-конфіги переплутані. `rviz/deliverable_2_3.rviz` насправді містить топіки Deliverable 4 (`/landmarks_*`, `/poses_*`), а `rviz/lab7.rviz` — топіки Deliverable 2/3 (`/gt_trajectory`, `/optimal_trajectory`, ...). Обидва `launch`-файли відкривають `lab7.rviz`, тож для Deliverable 4 запускайте RViz окремо: `rviz2 -d $(ros2 pkg prefix lab_7)/share/lab_7/rviz/deliverable_2_3.rviz` (або виправте `args` у `deliverable_4.launch.yaml`).

macOS: RViz2 потребує XQuartz і `DISPLAY=:0` (див. `ros2-docker/README.md`). Без GUI вузли все одно працюють і друкують результати в консоль.

6 Командний Deliverable 1 — вступ до GTSAM

Team Deliverable 1 (10 балів)

За hands-on гайдом GTSAM (до розділу 3 включно): 1a — odometry BetweenFactor; 1b — «GPS» UnaryFactor; 1c — навмисно неточне початкове наближення. Зберегти значення до/після оптимізації в deliverable_1.txt.

6.1 Теорія

Змінні — три пози $\text{Pose2 } x_k = (x, y, \theta)$. Фактори:

- **Prior** на x_1 : $e = \text{Log}(\mu^{-1}x_1)$ — «поза x_1 близька до μ ».
- **BetweenFactor** (одометрія): $e = \text{Log}(z_{k,k+1}^{-1} \cdot (x_k^{-1}x_{k+1}))$ — виміряне відносне переміщення z порівнюється з передбаченим $x_k^{-1}x_{k+1}$.
- **UnaryFactor** («GPS»): $e = (x - m_x, y - m_y)$ — вимірює лише позицію.

Якобіан UnaryFactor (уже написаний у стенсилі — варто розібратися чому). Ретракція Pose2 : $(x, y, \theta) \oplus (\delta_x, \delta_y, \delta_\theta)$ зсуває позицію на $R(\theta)(\delta_x, \delta_y)^\top$ (зсув у **локальній** системі робота). Отже

$$H = \partial e / \partial \delta = \begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \end{pmatrix} = [R(\theta) \mid \mathbf{0}]. \quad (32)$$

∂e :

μ	середнє prior-фактора (<code>priorMean = (0,0,0)</code>)
$z_{k,k+1}$	виміряна одометрія між x_k та x_{k+1} (тут $(2,0,0)$ — «2 м вперед»)
(m_x, m_y)	виміряна «GPS»-позиція
$R(\theta)$	матриця повороту 2×2 на кут курсу робота

6.2 Код

```
// Start of 1a.
graph.add(BetweenFactor<Pose2>(1, 2, odometry, odometryNoise));
graph.add(BetweenFactor<Pose2>(2, 3, odometry, odometryNoise));
// End of 1a.
```

```
// Start of 1b.
graph.add(UnaryFactor(1, 0.0, 0.0, unaryNoise));
graph.add(UnaryFactor(2, 2.0, 0.0, unaryNoise));
graph.add(UnaryFactor(3, 4.0, 0.0, unaryNoise));
// End of 1b.
```

```
// Start of 1c.
initial.insert(1, Pose2(0.5, 0.0, 0.2));
initial.insert(2, Pose2(2.3, 0.1, -0.2));
initial.insert(3, Pose2(4.1, 0.1, 0.1));
// End of 1c.
```

Змінна	Тип	Що означає
graph	NonlinearFactorGraph	контейнер факторів — наша сума 3
1, 2, 3	Key	ключі змінних (поз); у GTSAM це просто цілі числа
odometry	Pose2	виміряне відносне переміщення
odometryNoise, unaryNoise	SharedNoiseModel	сигми шуму: $(0.2, 0.2, 0.1)$ і $(0.1, 0.1)$

initial	Values	початкове наближення (лінеаризація стартує звідси)
---------	--------	--

6.3 Результат (реальний прогін)

Збережіть вивід: `ros2 run lab_7 deliverable_1 > deliverable_1.txt`.

Initial Estimate	Optimized Estimate
Value 1: (0.5, 0, 0.2)	Value 1: (1.4e-12, -3.9e-12, -4.3e-12)
Value 2: (2.3, 0.1, -0.2)	Value 2: (2, 6.6e-12, 1.9e-13)
Value 3: (4.1, 0.1, 0.1)	Value 3: (4, -2.2e-12, 1.9e-13)

Оптимум — рівно $(0, 0, 0)$, $(2, 0, 0)$, $(4, 0, 0)$: усі вимірювання взаємно узгоджені (одометрія «2 м вперед» і GPS $0 \rightarrow 2 \rightarrow 4$), тож існує розв'язок з нульовою похибкою. Залишки $\sim 10^{-12}$ — машинна точність.

7 Командний Deliverable 2 — 3D pose graph

Team Deliverable 2 (10 балів)

У deliverable_2_3.cpp: 2a — `BetweenFactor<Pose3>` для всіх вимірювань одометрії; 2b — `prior` на вузол 1. Скриншоти RViz після 1, 3 і 6 ітерацій Гаусса–Ньютона.

7.1 Що генерує стенсил

- Еталонна траєкторія — спіраль: $N = 500$ поз, радіус 2 м, два оберти (4π), підйом від 0 до 1 м; перша поза $(2, 0, 0)$.
- Одометрія: $z_i = T_i^{-1}T_{i+1}$ з шумом $\sigma_{\text{rot}} = \sigma_{\text{pos}} = 10^{-2}$ (у стенсилі це додається в **локальних** координатах через `Pose3::Expmap`).
- Початкове наближення — **інша** спіраль (3π , еліпс 1.2×0.5 , підйом до 2 м), навмисно далека.

7.2 Код

```
// Start of 2a
// measurements[i] is the relative pose between gt poses i and i+1, which
// are variables with keys i+1 and i+2 (keys start at 1).
for (size_t i = 0; i < measurements.size(); ++i) {
    graph.add(BetweenFactor<Pose3>(i + 1, i + 2, measurements[i], odometryNoise));
}
// End 2a.
```

```
// Start of 2b.
graph.add(PriorFactor<Pose3>(1, initial_pose, initialNoise));
// End 2b.
```

Змінна	Тип	Що означає
measurements	vector<Pose3>	499 зашумлених відносних поз z_i
i + 1, i + 2	Key	ключі сусідніх поз; нумерація змінних починається з 1, а вимірювань — з 0
odometryNoise	Diagonal	сигми в порядку Pose3: 3 обертальні, 3 позиційні
initial_pose	Pose3	$(I, (2, 0, 0))$ — збігається з першою еталонною позою
max_solver_iterations_	size_t	параметр ROS — скільки ітерацій GN дозволено

7.3 Результати (реальні прогони, без MoCap)

Ітерація	0 (старт)	1	2	3	5
$E(X)$	$\approx 7.8 \cdot 10^3$	$\approx 2.5 \cdot 10^3$	≈ 110	$\approx 10^{-4}$	$3 \cdot 10^{-25}$ (збіглося)

(Точні числа трохи різняться між запусками — шум генерується випадково.)

Що маєте побачити в RViz (`lab7.rviz`; числа нижче перевірені прогоном, картинку — ні, прогін був без GUI): зелена — еталон, біло-червона — зашумлена траєкторія, червона — початкове наближення, бірюзова — оптимізована.

- Після 1 ітерації бірюзова вже схожа на спіраль, але спотворена: задача сильно нелінійна через обертання (θ старту далекий від правильного на сотні градусів у кінці траєкторії).
- Після 3 ітерацій — практично збіглося.
- Після 6 — ідеальний збіг... **із зашумленою** траєкторією, а не з еталоном!

Примітка

Чому оптимум = зашумлена траєкторія. Граф — ланцюжок (дерево): 499 відносних факторів + 1 prior на 500 змінних. Для дерева завжди існує розв’язок, що задовольняє всі фактори точно (просто «проінтегрувати» одометрію від prior), тож $E^* = 0$ і оптимум — це dead reckoning. Без додаткової інформації (loop closure, GPS/MoCap) оптимізація не може прибрати накопичений дрейф. Це і мотивує Deliverable 3.

8 Командний Deliverable 3 — фактор Motion Capture

Team Deliverable 3 (15 балів)

За — реалізувати `MoCapPosition3Factor` (вимірювання — `Point3` позиції, змінна — `Pose3`) з якобіаном; 3b — шумова модель і додавання факторів. Скриншоти після 1, 3, 6 ітерацій з `use_mocap=true`.

8.1 Виведення фактора

Модель вимірювання: `MoCap` бачить **позицію** робота, $h(T) = t$. Похибка:

$$e(T) = t - m \in \mathbb{R}^3. \quad (33)$$

Якобіан за локальними координатами `Pose3` $\xi = (\omega, v)$: з 8 позиція після ретракції — $t + Rv$ (обертання ω позицію в першому порядку не змінює). Отже

$$H = \frac{\partial e(T \oplus \xi)}{\partial \xi} = [0_{3 \times 3} \mid R] \in \mathbb{R}^{3 \times 6}. \quad (34)$$

Це той самий `UnaryFactor` з T-1, лише в 3D: там було $[R(\theta) \mid 0]$, бо в `Pose2` порядок координат (x, y, θ) — позиція **спочатку**. У `Pose3` порядок навпаки, тому нулі зліва.

де:

m виміряна `MoCap`-позиція (`m_`)
 t, R позиція й обертання поточної оцінки пози `p`
 H 3×6 : нулі за обертанням, R за локальним зсувом

8.2 Код За — `include/deliverable_2_3.h`

```
MoCapPosition3Factor(gtsam::Key poseKey,
                     const gtsam::Point3& m,
                     const gtsam::SharedNoiseModel& model)
: NoiseModelFactor1<Pose3>(model, poseKey, m_(m)) {}

gtsam::Vector evaluateError(const gtsam::Pose3& p,
                           boost::optional<gtsam::Matrix&> H = boost::none) const {
    // За. Complete definition of factor
    // Measurement model h(T) = t (position of the pose). With GTSAM's Pose3
    // retraction T * Exp([w; v]) the position moves by R * v to first order,
    // so dh/d[w; v] = [0_{3x3}, R].
    if (H) {
        gtsam::Matrix36 J;
        J << gtsam::Matrix3::Zero(), p.rotation().matrix();
        *H = J;
    }
    return p.translation() - m_;
    // End За.
}
```

Порада

Той самий якобіан GTSAM дає вбудовано: `p.translation(H)` заповнює `OptionalJacobian<3,6>` рівно $[0 \mid R]$. Але для звіту варто написати явно — оцінюють саме виведення.

8.3 Код 3b — src/deliverable_2_3.cpp

```
// Start of 3b.
// MoCap measures a 3D position, so the noise model is 3-dimensional.
const noiseModel::Diagonal::shared_ptr mocapNoise =
    noiseModel::Diagonal::Sigmams(Vector3::Constant(mocap_std_dev));

// TODO: add the MoCap factors
// ...
for (const auto& [key, position] : mocap_measurements) {
    graph.add(MoCapPosition3Factor(key, position, mocapNoise));
}
// End 3b.
```

Змінна	Тип	Що означає
mocap_measurements	vector<pair<int, Point3>>	пари (ключ пози, виміряна позиція); 10 вимірювань — ключі 1, 51, ..., 451
mocap_std_dev	double	10^{-10} м — МоСар майже ідеальний
mocapNoise	Diagonal (3D)	три однакові сигми — по одній на x, y, z
key, position	int, Point3	structured binding з пари (C++17)

Prior не потрібен: 10 абсолютних позицій (неколінеарних) фіксують і положення, і орієнтацію всієї траєкторії — gauge знято. (У стенсилі prior додається лише при `use_mocap=false`.)

8.4 Результати (реальні прогони, з МоСар)

Ітерація	0	1	2	3	5	6
$E(X)$	$2.3 \cdot 10^{21}$	$1.6 \cdot 10^{21}$	$3.8 \cdot 10^{20}$	$5.5 \cdot 10^{19}$	$1.5 \cdot 10^{14}$	$2.3 \cdot 10^7$

Числа здаються величезними, бо $\sigma = 10^{-10}$ м дає вагу $1/\sigma^2 = 10^{20}$. Переведімо в метри: $E = 2.3 \cdot 10^7 \Rightarrow$ сумарний квадрат відхилень $2E\sigma^2 \approx 5 \cdot 10^{-13}$ м², тобто позиції в МоСар-точках збігаються з вимірюваннями до мікронів. Отже в RViz бірюзова траєкторія після 3–6 ітерацій має лягти на **зелену** (еталон), а не на зашумлену: МоСар прибирає дрейф.

9 Командний Deliverable 4 — bundle adjustment

Team Deliverable 4 (20 балів)

Розв'язати БА в deliverable_4.cpp: prior на x_0 , GenericProjectionFactor для кожного спостереження, prior на l_0 (масштаб). Звітуйте початкові та фінальні значення; скриншот RViz (landmarks і камери до/після + еталон).

9.1 Теорія

Змінні: пози камер $x_i = T_{C_i}^W \in \text{SE}(3)$ (камера $\rightarrow$ світ, як у GTSAM) і точки $l_j \in \mathbb{R}^3$. Спостереження — піксель u_{ij} точки j у камері i . Модель вимірювання (камера-обскура):

$$p_C = R_i^\top (l_j - t_i), \quad \pi(x_i, l_j) = \begin{pmatrix} f_x p_x / p_z + c_x \\ f_y p_y / p_z + c_y \end{pmatrix}, \quad e_{ij} = \pi(x_i, l_j) - u_{ij}. \quad (35)$$

БА мінімізує сумарну **похибку перепроєкції**:

$$\min_{\{x_i\}, \{l_j\}} \sum_{i,j} \|\pi(x_i, l_j) - u_{ij}\|_{\Sigma_u}^2, \quad \Sigma_u = \sigma_u^2 I_2. \quad (36)$$

Це ML-оцінка для гаусового піксельного шуму — кращого способу оцінити пози й точки за самими спостереженнями немає (на відміну від 5/8-точкових методів Lab 6, які мінімізують алгебраїчну похибку).

Gauge. Усі проєкційні фактори інваріантні до перетворення подібності світу (R, t і масштаб s — 7 ступенів свободи). Prior на x_0 фіксує 6, prior на l_0 — масштаб (відстань від камери 0 до точки 0) і ще 2 ступені, які й так зафіксовані — легке надвизначення, не шкодить.

де:

$x_i = (R_i, t_i)$	поза i -ї камери у світі (Symbol 'x', i)
l_j	3D-точка (landmark) у світі (Symbol 'l', j)
$p_C = (p_x, p_y, p_z)$	точка в системі камери
f_x, f_y, c_x, c_y	intrinsics Cal3_S2(50, 50, 0, 50, 50): фокус 50 px, центр (50, 50), skew 0
u_{ij}	вимірний піксель (kpt_coords_)
σ_u	піксельний шум, 1 px (measurementNoise)

9.2 Код

```
// TODO: Add a prior on pose x0. This indirectly specifies where the origin is.
// 0.3 rad on roll, pitch, yaw and 0.1 m on x, y, z
noiseModel::Diagonal::shared_ptr poseNoise =
    noiseModel::Diagonal::Sigmas((Vector(6) << Vector3::Constant(0.3),
                                   Vector3::Constant(0.1)).finished());
graph.add(PriorFactor<Pose3>(Symbol('x', 0), camera_poses[0], poseNoise));
```

```
// TODO: Add GenericProjectionFactor factors to the graph.
for (const auto& [cam_idx, kpt] : measurements) {
    graph.add(GenericProjectionFactor<Pose3, Point3, Cal3_S2>(
        kpt.kpt_coords_, measurementNoise, Symbol('x', cam_idx),
        Symbol('l', kpt.lmk_idx), K));
}
```

```
// TODO: ... add a prior on the position of the first landmark (fixes scale).
noiseModel::Isotropic::shared_ptr pointNoise = noiseModel::Isotropic::Sigma(3, 0.1);
graph.add(PriorFactor<Point3>(Symbol('l', 0), landmarks[0], pointNoise));
```

Змінна	Тип	Що означає
camera_poses	vector<Pose3>	8 еталонних камер на колі радіуса 3 м, усі дивляться в центр куба
landmarks	vector<Point3>	8 вершин куба ± 1 м
measurements	vector<pair<size_t, Keypoint>>	64 спостереження: (індекс камери, (індекс точки, піксель))
cam_idx, kpt	size_t, Keypoint	розпакована пара
Symbol('x', i)	Key	ключ з літерою: у 64-бітному ключі старший байт — символ, решта — номер
K	Cal3_S2::shared_ptr	калібрування камери
poseNoise, pointNoise	Diagonal, Isotropic	сигми р'іор-факторів («будь-які розумні» за завданням)

9.3 Результати (реальний прогін)

```

initial error = 12033.96
LM iterations: 604.2 → 8.31 → 3.7e-3 → 4.7e-10 → 1.6e-23
final error   = 1.56e-23
l1 covariance: diag ≈ (0.041, 0.098, 0.097)
l2 covariance: diag ≈ (0.111, 0.104, 0.566)

```

- Похибка падає до машинного нуля: спостереження згенеровані **без шуму**, тож еталон — точний розв'язок, і LM його знаходить (5 ітерацій).
- Маргінальні коваріації точок кажуть, наскільки кожную точку «визначено» графом. У l_2 дисперсія по z помітно більша (0.57) — ймовірно тому, що всі камери лежать у площині $z = 0$ і вертикальну координату точок визначено гірше.

RViz (див. пастку про конфіги в розділі 5): зелені — еталон, червоні — початкове наближення (точки зсунуті на $(-0.25, 0.2, 0.15)$, камери повернуті Родрігом $(-0.1, 0.2, 0.25)$), сині — після оптимізації, мають лягти на зелені.

10 Командний Deliverable 5 (опційний) — усереднення $SO(3)$

Team Deliverable 5 (+15 балів)

Реалізувати фактор, що кодує хордальну похибку $\|R - R_i\|_F$ між оцінкою й вимірним обертанням, побудувати граф, задати грубе початкове наближення, оптимізувати, порівняти.

10.1 Теорія

Це індивідуальний Deliverable 2 (пункти 1, 4, 5) для $SO(3)$: залишок $e_i(R) = \text{vec}(R - R_i) \in \mathbb{R}^9$, якобіан — J_R з 27. Сума $\sum_i \|e_i\|^2$ — хордальна функція втрат, оптимум — $\mathcal{P}_{SO(3)}(\sum R_i)$. Стенсил задає шум 9-вимірний з $\sigma = 10^{-2}$ — розмірність похибки має збігатися з розмірністю шумової моделі.

10.2 Код — include/deliverable_5.h

```
// Unary factor with the 9-dimensional error vec(R - R_meas) (column-major), so
// that the squared error is the chordal distance ||R - R_meas||_F^2.
class FrobeniusNormFactor : public NoiseModelFactor1<Rot3> {
private:
    Rot3 measured_;

public:
    FrobeniusNormFactor(Key key, const Rot3& measured, const SharedNoiseModel& model)
        : NoiseModelFactor1<Rot3>(model, key), measured_(measured) {}

    Vector evaluateError(const Rot3& R, boost::optional<Matrix&> H = boost::none) const
    override {
        const Matrix3 Rm = R.matrix();
        if (H) {
            // Rot3 retraction: R * Exp(w) ~ R (I + [w]x), so d vec(R Exp(w)) / dw_k
            // = vec(R [e_k]x). Columns of R [e_k]x: [0, r3, -r2], [-r3, 0, r1], [r2, -r1, 0].
            const Vector3 r1 = Rm.col(0), r2 = Rm.col(1), r3 = Rm.col(2);
            Matrix J = Matrix::Zero(9, 3);
            J.block<3, 1>(3, 0) = r3;
            J.block<3, 1>(6, 0) = -r2;
            J.block<3, 1>(0, 1) = -r3;
            J.block<3, 1>(6, 1) = r1;
            J.block<3, 1>(0, 2) = r2;
            J.block<3, 1>(3, 2) = -r1;
            *H = J;
        }
        const Matrix3 D = Rm - measured_.matrix();
        return Eigen::Map<const Vector9>(D.data());
    }
};
```

Примітка

Eigen зберігає матриці по стовпцях, тож `Eigen::Map<const Vector9>(D.data())` — це саме $\text{vec}(D)$ по стовпцях, узгоджений з рядками якобіана: рядки 0–2 — перший стовпець R , 3–5 — другий, 6–8 — третій.

10.3 Код — src/deliverable_5.cpp

```
NonlinearFactorGraph graph;
// ... (шумова модель noise вже є в стенсилі)
for (const Rot3& R_i : measurements) {
```

```

graph.add(FrobeniusNormFactor(1, R_i, noise));
}
graph.print("Factor graph:\n");
// ... (initial вже є в стенсилі: вимірювання 0, повернуте на 45° навколо y)
initial.print("Initial estimate:\n");

LevenbergMarquardtParams params;
params.setVerbosity("ERROR");
const Values result = LevenbergMarquardtOptimizer(graph, initial, params).optimize();
result.print("Final result:\n");
std::cout << "initial error = " << graph.error(initial) << std::endl;
std::cout << "final error = " << graph.error(result) << std::endl;

// Closed form for comparison: projection of the mean onto SO(3).
Matrix3 M = Matrix3::Zero();
for (const Rot3& R_i : measurements) M += R_i.matrix();
Eigen::JacobiSVD<Matrix3> svd(M, Eigen::ComputeFullU | Eigen::ComputeFullV);
Matrix3 S = Matrix3::Identity();
S(2, 2) = (svd.matrixU() * svd.matrixV().transpose()).determinant();
const Rot3 closed_form(svd.matrixU() * S * svd.matrixV().transpose());
std::cout << "angle(GTSAM, closed form) = "
            << result.at<Rot3>(1).between(closed_form).axisAngle().second << " rad\n";

```

Змінна	Тип	Що означає
measurements	vector<Rot3>	10 зашумлених обертань навколо I ($\sigma = 10^{-2}$ рад)
noise	Diagonal (9D)	по одній сигмі на кожен елемент матриці
1	Key	єдина змінна — шукане середнє обертання
closed_form	Rot3	$\mathcal{P}_{SO(3)}(\sum R_i)$ для перевірки

10.4 Результати (реальний прогін)

```

Initial estimate: R ≈ Ry(45°) · R_0      initial error = 57463.7
Final result:    R ≈ I (відхилення ~0.4°) final error   = 31.59
LM: 4 ітерації
angle(GTSAM, closed form) = 8.8e-13 rad

```

Навіть зі старту за 45° LM сходиться за 4 ітерації до **точно** того самого обертання, що й замкнена формула (розбіжність 10^{-12} рад — машинна точність). Фінальна похибка не нульова — це залишковий розкид самих вимірювань навколо їх середнього.

11 Чекліст здачі

- **Індивідуально (Gradescope, PDF):** I-1 — пункти (1)–(3) з розділу 3; I-2 — (1)–(4) з розділу 4, бонус (5) — явний J_R .
- **Команда (папка lab7 у репозиторії):** увесь пакет з кодом + PDF з:
 - T-1: deliverable_1.txt (початкові й фінальні значення 3 поз);
 - T-2: 3 скриншоти RViz (1, 3, 6 ітерацій, use_mocap:=false);
 - T-3: 3 скриншоти RViz (1, 3, 6 ітерацій, use_mocap:=true);
 - T-4: початкові й фінальні значення точок і поз + скриншот RViz;
 - T-5 (опц.): початкове й фінальне обертання, похибки.

Корисні команди для збору виводу:

```
ros2 run lab_7 deliverable_1 > deliverable_1.txt
ros2 launch lab_7 deliverable_2_3.launch.yaml max_solver_iterations:=1 use_mocap:=false
ros2 launch lab_7 deliverable_2_3.launch.yaml max_solver_iterations:=6 use_mocap:=true
ros2 run lab_7 deliverable_4 2>&1 | tee deliverable_4.txt # Ctrl+C після "final error"
ros2 run lab_7 deliverable_5 > deliverable_5.txt
```

А Словник позначень

Символ	Значення	Розділ
X, X_i	усі змінні графа / змінні i -го фактора	2.1
z_i, h_i	вимірювання та модель вимірювання	2.1
$\Sigma_i, \ \cdot\ _\Sigma$	коваріація шуму, махаланобісова норма	2.1
$r(x), J$	вектор залишків, яacobіан	2.2
δ, λ, D	крок оптимізатора, демпфування LM, масштабування	2.2
W, L	вагова матриця, множник Холецкого $W = LL^\top$	3
$\tilde{r}, \tilde{J}$	вибілені залишок і яacobіан	3
R, t, T	обертання, трансляція, поза SE(3)	1.1
ω, v, ξ	локальні координати: обертання, зсув, $[\omega; v]$	2.3
$\oplus, \text{Exp}, \text{Log}$	ретракція, експонента й логарифм групи	2.3
H	яacobіан фактора за локальними координатами (<code>evaluateError</code>)	2.3
$E(X)$	<code>graph.error</code> — $\frac{1}{2} \sum \ e\ _\Sigma^2$	2.5
T_i, R_i, t_i	вимірювання для усереднення	4
M, U, S, V	сума обертань та її SVD (у 4.1); (!) у 2.3 немає	4.1
$\mathcal{P}_{\text{SO}(3)}$	проекція на SO(3)	4.1
ω_i, ρ_i	ваги (концентрація Ланжевена, обернена дисперсія трансляції)	4.2
Ω_i	вагова матриця 4×4 зваженої хордальної норми	4.2
$c(\omega)$	нормувальна константа Ланжевена	4.3
J_R, r_k	яacobіан $\text{vec}(R \text{Exp}(\omega))$; стовпці R	4.4, 4.5
$A_i, w(A)$	$R^\top R_i$ і вектор його антисиметричної частини	4.4
x_i, l_j	пози камер і точки в ВА	9
u_{ij}, π	виміряний піксель і функція проєкції	9
σ_u	піксельний шум	9

Джерела: завдання курсу VNAV (MIT, CC BY 4.0); F. Dellaert, *Factor Graphs and GTSAM: A Hands-on Introduction*; GTSAM 4.2 source; конспекти VNAV, лекції 15–16.